

我在 CS336 中实现 Byte-level BPE Tokenizer 的完整记录

ln

2026 年 3 月 20 日

目录

1 写在前面	1
2 目标与整体结构	1
3 实现中用到的工具	2
3.1 Python 标准库	2
3.2 第三方库	2
3.3 核心数据结构	2
4 重要函数与接口	2
4.1 special token 相关	2
4.2 pretokenization 相关	3
4.3 BPE 相关	3
4.4 训练相关	3
4.5 类接口	4
5 测试接口是如何连接的	5
5.1 get_tokenizer	5
5.2 run_train_bpe	5
6 我做的训练优化	6
6.1 朴素实现的问题	6
6.2 优化后的思路	6
6.3 为什么它是正确的	6
6.4 优化效果	6
7 一个完整的 example: 从输入文本到 token ids	7
7.1 初始化时发生了什么	7
7.2 第 1 层: split_special_tokens	7
7.3 第 2 层: iter_text_units	7
7.4 第 3 层到第 5 层: 逐个 pretoken 的字节拆分、merge 和 id 映射	8
7.5 更细的内部 trace: hello	9

7.6 更细的内部 trace: <code>Talkking</code>	10
7.7 更细的内部 trace: <code>Attention</code>	11
7.8 最终编码结果	12
8 训练路径的调用栈	12
9 测试命令与结果	12
9.1 语法检查	12
9.2 速度测试	12
9.3 完整 tokenizer 测试	13
10 我对这份实现的总结	13

1 写在前面

这份报告记录了我在 Stanford CS336 Assignment 1 中实现 **byte-level BPE tokenizer** 的全过程。我的目标不是只把测试跑通，而是把整个实现路径讲清楚：为什么这样拆函数、接口是怎样接上的、训练为什么需要优化、以及一段真实文本在我的实现里到底经历了哪些函数调用和状态变化。

本次实现对应的核心代码位于：

- `cs336/codepart/part1_components/assignment1-basics/cs336_basics/tokenizer.py`
- `cs336/codepart/part1_components/assignment1-basics/tests/adapters.py`

2 目标与整体结构

我实现的是一个典型的 GPT-2 风格 tokenizer。它有三条主线：

1. **构造器与加载**：从 GPT-2 风格的 `vocab.json` 和 `merges.txt` 恢复出内部状态。
2. **编码**：把字符串变成 token id 序列。
3. **训练**：从语料中训练出新的 BPE vocabulary 和 merge 列表。

我的实现把这三条主线拆成了几层：

- **文本层**：`split_special_tokens`、`pretokenize_text`、`iter_text_units`
- **字节层**：`pretoken_to_byte_tokens`、`bytes_to_byte_tokens`
- **BPE 层**：`get_adjacent_pairs`、`merge_pair_in_sequence`、`merge_pretoken`
- **训练层**：`_collect_pretoken_counts`、`_build_pair_statistics`、`train_bpe`
- **接口层**：`Tokenizer.from_files`、`Tokenizer.encode`、`Tokenizer.encode_iterable`、`Tokenizer.decode`

3 实现中用到的工具

3.1 Python 标准库

- `Counter`: 统计 pretoken 序列和 pair 频次
- `defaultdict(set)`: 建立 pair -> sequences 的反向索引
- `lru_cache`: 缓存 GPT-2 的 byte 到 printable unicode 的映射表
- `json / os.PathLike`: 从 GPT-2 风格文件中恢复 tokenizer

3.2 第三方库

- `regex`: 用于 GPT-2 handout 里给出的正则表达式; 标准库 `re` 不支持 `\p{L}`、`\p{N}`
- `pytest`: 驱动课程提供的测试

3.3 核心数据结构

- `vocab`: `dict[int, bytes]`: id 到 token bytes 的映射
- `token_to_id`: `dict[bytes, int]`: 反向映射
- `merges`: `list[tuple[bytes, bytes]]`: merge 的创建顺序
- `merge_ranks`: `dict[tuple[bytes, bytes], int]`: pair 到 rank 的映射, rank 越小优先级越高
- `token_counts`: `Counter[tuple[bytes, ...]]`: 训练时每个 pretoken 序列的频次
- `pair_counts`: `Counter[tuple[bytes, bytes]]`: 训练时所有相邻 pair 的全局频次
- `pair_to_sequences`: `dict[pair, set[sequence]]`: 训练优化时的反向索引

4 重要函数与接口

4.1 special token 相关

`normalize_special_tokens` 输入是 `Sequence[str] | None`, 输出是一个规范化后的列表。它做两件事:

1. 去重
2. 按 `(-len(token), token)` 排序

排序规则的目的是优先匹配更长的 special token, 这样才能正确处理重叠 special token。

`build_special_token_regex` 把 special token 列表转成一个可复用的正则对象; 如果 special token 为空, 则返回 `None`。

split_special_tokens 先把原始文本按 special token 切开，输出格式是：

```
[(is_special, segment), ...]
```

4.2 pretokenization 相关

pretokenize_text 使用 handout 指定的 GPT-2 正则：

```
'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[\s\p{L}\p{N}]+\s+(?!S)|\s+
```

把普通文本拆成 pretoken。

iter_text_units 这是 encode 的直接上游函数。它先调用 **split_special_tokens**，然后对 ordinary segment 再调用 **pretokenize_text**，最终统一输出：

```
(True, special_token) / (False, pretoken)
```

4.3 BPE 相关

pretoken_to_byte_tokens 把一个 pretoken 先做 UTF-8 编码，再拆成单字节 token。比如：

```
"hello" -> b"hello" -> [b"h", b"e", b"l", b"l", b"o"]
```

get_adjacent_pairs 给定一个 token 序列，返回所有相邻 pair。例如：

```
[b"h", b"e", b"l"] -> [(b"h", b"e"), (b"e", b"l")]
```

merge_pair_in_sequence 把一个指定 pair 在序列中所有从左到右、非重叠的出现都 merge 掉。例如：

```
[b"a", b"b", b"a", b"b"], pair=(b"a", b"b")
-> [b"ab", b"ab"]
```

merge_pretoken 这是编码阶段最关键的函数。它会反复执行：

1. 找当前序列中所有相邻 pair
2. 选出 rank 最小的可 merge pair
3. 调用 **merge_pair_in_sequence** 合并
4. 直到没有可 merge 的 pair

4.4 训练相关

_collect_pretoken_counts 先把训练语料变成：

```
Counter[tuple[bytes, ...]]
```

也就是每个 pretoken 字节序列的出现频次。这里 special token 只当边界，不参与 merge 学习。

_sequence_pair_counts 统计单个 token sequence 内部每个 pair 出现了几次。

_build_pair_statistics 从 token_counts 一次性构建：

- pair_counts
- pair_to_sequences

_remove_sequence_from_pair_index 当一个旧序列已经被 merge 生成新序列时，把旧序列从反向索引里移除，防止索引污染。

train_bpe 训练主循环的逻辑是：

1. 初始化 256 个单字节 token
2. 加入 special tokens
3. 收集 token_counts
4. 建立 pair_counts 与 pair_to_sequences
5. 每轮选一个 best_pair
6. 只更新受影响的序列
7. 更新 vocab 和 merges

4.5 类接口

Tokenizer.from_files 从 GPT-2 风格文件中恢复 tokenizer。这里需要先用 gpt2_bytes_to_unicode 的逆映射，把 printable unicode 再转回真实 bytes。

Tokenizer.encode 编码主链路如下：

```
Tokenizer.encode
-> iter_text_units
    -> split_special_tokens
    -> pretokenize_text
-> _encode_pretoken_to_ids
    -> _encode_pretoken_to_bytes
        -> merge_pretoken
            -> pretoken_to_byte_tokens
            -> get_adjacent_pairs
            -> merge_pair_in_sequence
-> token_to_id lookup
```

Tokenizer.encode_iterable 流式编码接口。它使用一个 buffer 保存 chunk 边界上的尾巴，并通过 _split_stable_ordinary_tail 确保 chunk 切分不会改变编码结果。

Tokenizer.decode 解码相对直接:

```
ids -> vocab lookup -> b"".join(...) -> decode("utf-8", errors="replace")
```

5 测试接口是如何连接的

课程测试不直接 import 我的实现, 而是统一通过 tests/adapters.py 调用。因此, 我只需要把 adapter 接到自己的实现上。

5.1 get_tokenizer

```
def get_tokenizer(
    vocab: dict[int, bytes],
    merges: list[tuple[bytes, bytes]],
    special_tokens: list[str] | None = None,
) -> Any:
    from cs336_basics.tokenizer import Tokenizer
    return Tokenizer(vocab=vocab, merges=merges, special_tokens=special_tokens)
```

这条链路是:

```
tests/test_tokenizer.py
-> tests/adapters.py::get_tokenizer
-> cs336_basics.tokenizer.Tokenizer(...)
```

5.2 run_train_bpe

```
def run_train_bpe(
    input_path: str | os.PathLike,
    vocab_size: int,
    special_tokens: list[str],
    **kwargs,
) -> tuple[dict[int, bytes], list[tuple[bytes, bytes]]]:
    from cs336_basics.tokenizer import train_bpe
    return train_bpe(
        input_path=input_path,
        vocab_size=vocab_size,
        special_tokens=special_tokens,
    )
```

这条链路是:

```
tests/test_train_bpe.py
-> tests/adapters.py::run_train_bpe
-> cs336_basics.tokenizer.train_bpe(...)
```

6 我做的训练优化

6.1 朴素实现的问题

最朴素的 `train_bpe` 有两个明显瓶颈：

1. 每一轮 merge 都重新扫描整个 `token_counts`，重建全部 `pair_counts`
2. 每一轮都把所有 `sequence` 重新尝试 merge，即使它根本不包含当前的 `best_pair`

这种做法在小样本上没问题，但在课程的速度测试里会超时。

6.2 优化后的思路

我把训练改成了 **增量更新**：

1. 初始化时一次性构建 `pair_counts` 和 `pair_to_sequences`
2. 每轮先选出 `best_pair`
3. 通过 `pair_to_sequences[best_pair]` 找到真正受影响的序列
4. 先移除这些旧序列的 `pair` 贡献
5. 对这些旧序列执行 merge，生成新序列
6. 再把新序列的 `pair` 贡献加回去

换句话说，优化并没有改变算法语义，只是避免了每轮全表重算。

6.3 为什么它是正确的

因为每轮我都做了这三步：

1. 从全局统计里删掉旧序列的贡献
2. 生成 merge 后的新序列
3. 把新序列的贡献加回全局统计

所以在任何时刻，`pair_counts` 和 `pair_to_sequences` 都等价于“如果把当前所有 `sequence` 从头再统计一遍”得到的结果。

6.4 优化效果

优化前，`test_train_bpe_speed` 大约在 2 秒量级，超过课程要求。优化后我重新运行：

```
./venv/bin/python -m pytest tests/test_train_bpe.py::test_train_bpe_speed
```

结果是：

```
tests/test_train_bpe.py::test_train_bpe_speed PASSED
===== 1 passed in 0.28s =====
```

7 一个完整的 example: 从输入文本到 token ids

这一节我不再用抽象例子, 而是直接用下面这段真实文本:

```
hello, world! Talkking is cheap, show me your code, coding is cheap show me your prompt.  
Attention is all you need.
```

这里我使用的是 `tests/fixtures/gpt2_vocab.json` 和 `tests/fixtures/gpt2_merges.txt` 恢复出的 tokenizer, 也就是课程测试里同样使用的 GPT-2 风格词表。

7.1 初始化时发生了什么

调用:

```
tok = Tokenizer.from_files(  
    "tests/fixtures/gpt2_vocab.json",  
    "tests/fixtures/gpt2_merges.txt",  
)
```

我实际读到的初始化结果是:

```
vocab_size = 50257  
merges_size = 50000  
special_tokens = []  
_special_split_re = None  
_special_overlap = -1  
merge_ranks_size = 50000
```

也就是说, 这个 example 里:

- 没有 special token
- 因此 `split_special_tokens` 不会切出特殊片段
- `encode` 将完全走 ordinary text 路径

7.2 第 1 层: `split_special_tokens`

因为这里没有 special token, 所以它的输出只有一段:

```
[(False, 'hello, world! Talkking is cheap, show me your code, coding is cheap show me  
your prompt. Attention is all you need.')] ]
```

7.3 第 2 层: `iter_text_units`

接下来 `iter_text_units` 会调用 `pretokenize_text`, 把整段 ordinary text 切成 27 个 pre-token:

```
0 (False, 'hello')  
1 (False, ',')  
2 (False, ' world')
```



```

3 (False, '!')
4 (False, ' Talkking')
5 (False, ' is')
6 (False, ' cheap')
7 (False, ',')
8 (False, ' show')
9 (False, ' me')
10 (False, ' your')
11 (False, ' code')
12 (False, ',')
13 (False, ' coding')
14 (False, ' is')
15 (False, ' cheap')
16 (False, ' show')
17 (False, ' me')
18 (False, ' your')
19 (False, ' prompt')
20 (False, '.')
21 (False, ' Attention')
22 (False, ' is')
23 (False, ' all')
24 (False, ' you')
25 (False, ' need')
26 (False, '.')

```

7.4 第 3 层到第 5 层: 逐个 pretoken 的字节拆分、merge 和 id 映射

下表给出了每个 pretoken 的实际处理结果。

idx	pretoken	byte tokens	merged bytes	ids
0	hello	[b'h', b'e', b'l', b'l', b'o']	[b'hello']	[31373]
1	,	[b',']	[b',']	[11]
2	world	[b' ', b'w', b'o', b'r', b'l', b'd']	[b' world']	[995]
3	!	[b '!']	[b '!']	[0]
4	Talkking	[b' ', b'T', b'a', b'l', b'k', b'k', b'i', b'n', b'g']	[b' Talk', b'king']	[12167, 3364]
5	is	[b' ', b'i', b's']	[b' is']	[318]
6	cheap	[b' ', b'c', b'h', b'e', b'a', b'p']	[b' cheap']	[7026]
7	,	[b',']	[b',']	[11]
8	show	[b' ', b's', b'h', b'o', b'w']	[b' show']	[905]

idx	pretoken	byte tokens	merged bytes	ids
9	me	[b' ', b'm', b'e']	[b' me']	[502]
10	your	[b' ', b'y', b'o', b'u', b'r']	[b' your']	[534]
11	code	[b' ', b'c', b'o', b'd', b'e']	[b' code']	[2438]
12	,	[b',']	[b',']	[11]
13	coding	[b' ', b'c', b'o', b'd', b'i', b'n', b'g']	[b' coding']	[19617]
14	is	[b' ', b'i', b's']	[b' is']	[318]
15	cheap	[b' ', b'c', b'h', b'e', b'a', b'p']	[b' cheap']	[7026]
16	show	[b' ', b's', b'h', b'o', b'w']	[b' show']	[905]
17	me	[b' ', b'm', b'e']	[b' me']	[502]
18	your	[b' ', b'y', b'o', b'u', b'r']	[b' your']	[534]
19	prompt	[b' ', b'p', b'r', b'o', b'm', b'p', b't']	[b' prompt']	[6152]
20	.	[b'.']	[b'.']	[13]
21	Attention	[b' ', b'A', b't', b't', b'e', b'n', b't', b'i', b'o', b'n']	[b' Attention']	[47406]
22	is	[b' ', b'i', b's']	[b' is']	[318]
23	all	[b' ', b'a', b'l', b'l']	[b' all']	[477]
24	you	[b' ', b'y', b'o', b'u']	[b' you']	[345]
25	need	[b' ', b'n', b'e', b'e', b'd']	[b' need']	[761]
26	.	[b'.']	[b'.']	[13]

这个表能直接看出几个现象:

- GPT-2 风格 tokenizer 会把空格和后面的词绑在一起, 例如 ' world'、' cheap'、' Attention'
- Talkking 这种不规范拼写没有直接合成一个 token, 而是被拆成了 b' Talk' 与 b'king'
- 标点 , ! . 直接以单字节形式进入词表

7.5 更细的内部 trace: hello

下面我把 merge_pretoken 内部每一轮的变化完整写出来。

```
unit='hello'
pretoken_to_byte_tokens -> [b'h', b'e', b'l', b'l', b'o']
```

```

step 0 candidate_pairs -> [(b'h', b'e'), (b'e', b'l'), (b'l', b'l'), (b'l', b'o')]
step 0 mergeable_pairs -> [((b'h', b'e'), 2), ((b'e', b'l'), 161), ((b'l', b'l'), 41), ((
    b'l', b'o'), 5183)]
step 0 best_pair -> (b'h', b'e'), rank=2
step 0 after merge_pair_in_sequence -> [b'he', b'l', b'l', b'o']

step 1 candidate_pairs -> [(b'he', b'l'), (b'l', b'l'), (b'l', b'o')]
step 1 mergeable_pairs -> [((b'he', b'l'), 2722), ((b'l', b'l'), 41), ((b'l', b'o'),
    5183)]
step 1 best_pair -> (b'l', b'l'), rank=41
step 1 after merge_pair_in_sequence -> [b'he', b'll', b'o']

step 2 candidate_pairs -> [(b'he', b'll'), (b'll', b'o')]
step 2 mergeable_pairs -> [((b'he', b'll'), 12502), ((b'll', b'o'), 18542)]
step 2 best_pair -> (b'he', b'll'), rank=12502
step 2 after merge_pair_in_sequence -> [b'hell', b'o']

step 3 candidate_pairs -> [(b'hell', b'o')]
step 3 mergeable_pairs -> [((b'hell', b'o'), 31117)]
step 3 best_pair -> (b'hell', b'o'), rank=31117
step 3 after merge_pair_in_sequence -> [b'hello']

final merged parts -> [b'hello']
token_to_id lookup -> [31373]

```

7.6 更细的内部 trace: Talkking

这个例子比 `hello` 更有代表性, 因为它不会 `merge` 到单一 `token`, 而是停在两个 `token` 上。

```

unit=' Talkking'
pretoken_to_byte_tokens -> [b' ', b'T', b'a', b'l', b'k', b'k', b'i', b'n', b'g']

step 0 best_pair -> (b'i', b'n'), rank=3
after -> [b' ', b'T', b'a', b'l', b'k', b'k', b'in', b'g']

step 1 best_pair -> (b'in', b'g'), rank=22
after -> [b' ', b'T', b'a', b'l', b'k', b'k', b'ing']

step 2 best_pair -> (b'a', b'l'), rank=26
after -> [b' ', b'T', b'al', b'k', b'k', b'ing']

step 3 best_pair -> (b' ', b'T'), rank=53
after -> [b' T', b'al', b'k', b'k', b'ing']

step 4 best_pair -> (b'al', b'k'), rank=715
after -> [b' T', b'alk', b'k', b'ing']

```

```
step 5 best_pair -> (b'k', b'ing'), rank=3108
after -> [b' T', b'alk', b'king']

step 6 best_pair -> (b' T', b'alk'), rank=11911
after -> [b' Talk', b'king']

step 7 no mergeable pair, stop
final merged parts -> [b' Talk', b'king']
token_to_id lookup -> [12167, 3364]
```

7.7 更细的内部 trace: Attention

```
unit=' Attention'
pretoken_to_byte_tokens -> [b' ', b'A', b't', b't', b'e', b'n', b't', b'i', b'o', b'n']

step 0 best_pair -> (b'o', b'n'), rank=5
after -> [b' ', b'A', b't', b't', b'e', b'n', b't', b'i', b'on']

step 1 best_pair -> (b'e', b'n'), rank=12
after -> [b' ', b'A', b't', b't', b'en', b't', b'i', b'on']

step 2 best_pair -> (b'i', b'on'), rank=39
after -> [b' ', b'A', b't', b't', b'en', b't', b'ion']

step 3 best_pair -> (b'en', b't'), rank=42
after -> [b' ', b'A', b't', b't', b'ent', b'ion']

step 4 best_pair -> (b' ', b'A'), rank=61
after -> [b' A', b't', b't', b'ent', b'ion']

step 5 best_pair -> (b't', b't'), rank=670
after -> [b' A', b'tt', b'ent', b'ion']

step 6 best_pair -> (b'ent', b'ion'), rank=1207
after -> [b' A', b'tt', b'ention']

step 7 best_pair -> (b' A', b'tt'), rank=3204
after -> [b' Att', b'ention']

step 8 best_pair -> (b' Att', b'ention'), rank=47150
after -> [b' Attention']

final merged parts -> [b' Attention']
token_to_id lookup -> [47406]
```

7.8 最终编码结果

整段文本的最终 id 序列是：

```
[31373, 11, 995, 0, 12167, 3364, 318, 7026, 11, 905, 502, 534, 2438, 11, 19617, 318, 7026, 905, 502, 534, 6152, 13, 47406, 318, 477, 345, 761, 13]
```

再调用 decode 之后：

```
hello, world! Talkking is cheap, show me your code, coding is cheap show me your prompt.
Attention is all you need.
```

说明 round-trip 是正确的。

8 训练路径的调用栈

虽然上面的 example 主要展示的是编码，但训练路径同样重要。我的训练调用栈是：

```
run_train_bpe
-> train_bpe
  -> normalize_special_tokens
  -> _collect_pretoken_counts
    -> iter_text_units
      -> split_special_tokens
      -> pretokenize_text
    -> pretoken_to_byte_tokens
  -> _build_pair_statistics
  -> _sequence_pair_counts
-> while loop:
  -> choose best_pair from pair_counts
  -> _remove_sequence_from_pair_index
  -> merge_pair_in_sequence
  -> _sequence_pair_counts
-> return vocab, merges
```

9 测试命令与结果

9.1 语法检查

```
./venv/bin/python -m py_compile cs336_basics/tokenizer.py
```

这条命令没有输出，说明文件能被正常编译。

9.2 速度测试

```
./venv/bin/python -m pytest tests/test_train_bpe.py::test_train_bpe_speed
```

结果：

```
tests/test_train_bpe.py::test_train_bpe_speed PASSED
===== 1 passed in 0.28s =====
```

9.3 完整 tokenizer 测试

```
./venv/bin/python -m pytest tests/test_tokenizer.py tests/test_train_bpe.py
```

结果：

```
===== 27 passed, 1 xfailed in 12.05s =====
```

这里的 `xfailed` 是测试本身标记的预期失败项，不是我的实现出错。

10 我对这份实现的总结

从工程角度看，这份 tokenizer 的关键不是把 `encode` 写出来，而是同时把下面几件事处理干净：

1. 统一使用 `bytes` 作为内部 token 表示
2. special token 只做边界，不参与 merge 学习
3. 编码时严格按 merge rank 从小到大反复应用
4. 训练时通过增量更新避免每轮全量重算
5. 通过 adapter 把课程测试接口和自己的实现解耦

如果只看 API，这个 tokenizer 其实很简单：`from_files`、`encode`、`encode_iterable`、`decode`。但如果真的把它从 0 实现出来，就会发现它本质上是一个分层很清晰的系统：

- 文本分段层决定边界
- 字节层决定最小表示单位
- BPE 层决定 merge 规则
- 训练层决定如何得到这些规则
- adapter 层决定它怎样被测试框架调用

对我来说，这一版实现最大的收获不是“我写了一个 tokenizer”，而是我第一次把 **字符串处理、字节级建模、贪心 merge、增量统计和测试接线** 这几件原本分散的事情，连成了一条完整的工程链路。